

2DX3 Final Project 3D Scanning System

(Sam Chan, chans134, 400575501)

Features

This embedded system is a low-cost 3D scanning device that utilizes a Time-Of-Flight (ToF) sensor mounted on a stepper motor, which is driven by the MSP432E401Y microcontroller.



Figure 1: Physical design of device.

Key features include:

MSP432E401Y Microcontroller

- Central point of communication between ToF, stepper motor, and laptop

- Handles I2C communication with ToF sensor
- Handles UART communication with MATLAB for data transfer
- Controls stepper motor rotation in fixed 11.25-degree increments
- Converts raw measurements to Cartesian coordinates
- 14MHz bus speed

VL53L1X ToF Sensor

- Measures object distance through I2C and outputs distance readings in millimetres
- Maximum operating range of up to 4m in long-distance mode
- Fast ranging frequency of up to 50Hz [1]
- 5V operating voltage

Stepper Motor/ULN2003 Driver Board

- Full phase motor rotation
- 512 steps for a full 360-degree rotation
- Uses 16 motor steps per 11.25-degree scan increment
- 5V operating voltage

Serial Communication

- I2C for communication between microcontroller and ToF sensor
- UART for communication between microcontroller and laptop
- 115200 bps baud rate

3D Visualization Component

- MATLAB is the software used on the receiving side for visualizing distance measurements taken by ToF

General Description

This device provides a simple method to scan the physical environment in real-time, before showing it in a 3D manner. A ToF sensor is mounted on a stepper motor and its movement controlled by the microcontroller. As the motor rotates, the sensor collects distance measurements in the form of “slices”, taking in data from the YZ plane at angular intervals of 11.25 degrees. Whenever a distance reading is taken (initialized by PJ0, the onboard push button), the onboard LED D3 (PF4) turns on to indicate that a measurement is being performed. Additionally, upon each scan, another onboard LED D2 (PN0) flashes to indicate that the index (out of 32 scans) is increasing and the current scan for the specific angle has completed. Since the VL53L1X already contains its own internal signal conditioning and analog-to-digital conversion circuitry, the microcontroller receives processed distance data digitally through I2C rather than through its own ADC.

After one full set of scans is completed, the embedded program preprocesses the raw polar data by converting each angle-distance pair into Cartesian coordinates. These processed coordinates are then transmitted to an external computer through UART serial communication at 115200 bps. On the receiver side, MATLAB acts as the PC application by reading the incoming serial data, organizing the coordinate arrays, and graphically reconstructing the scanned environment as a 3D plot.

Data Flow Diagram

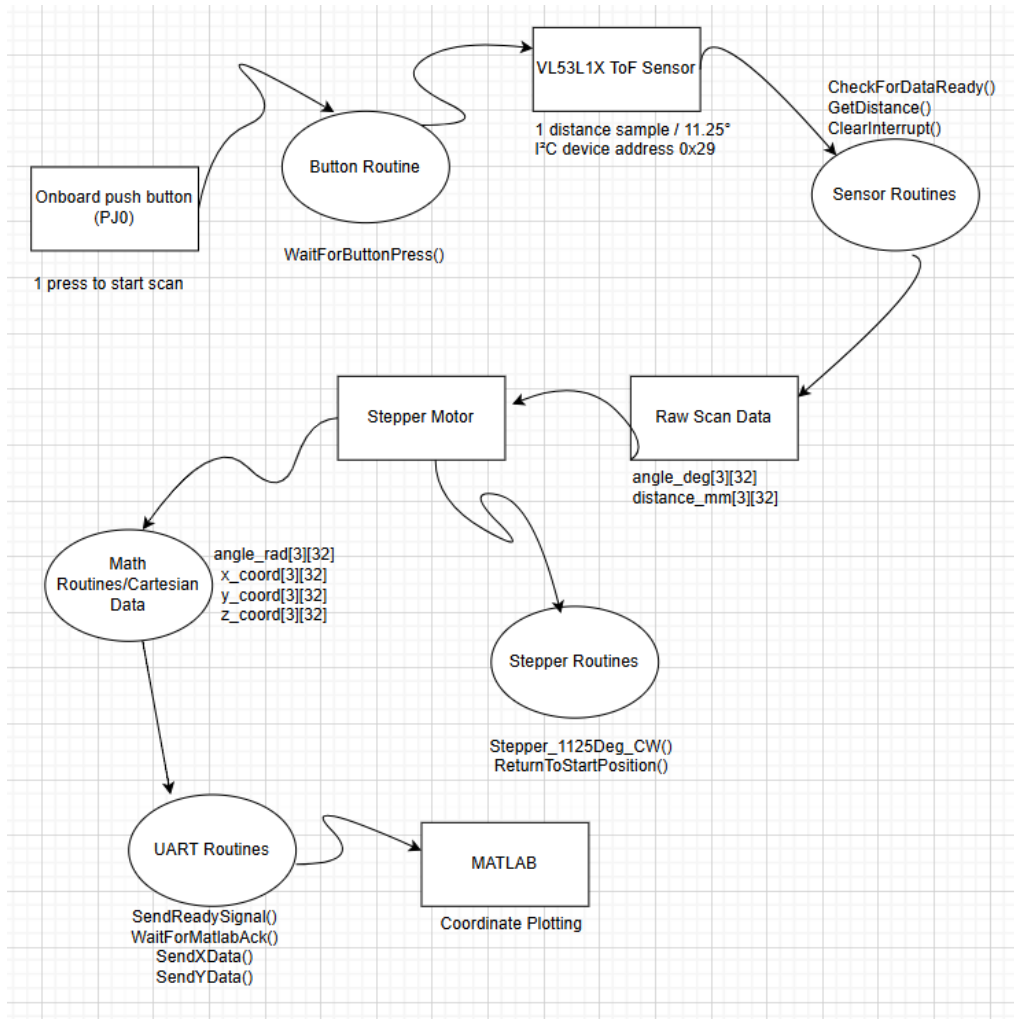


Figure 2: Data Flow diagram of embedded system.

Device Characteristics Table

Microcontroller (MSP432E401Y)		VL53L1X ToF Sensor		Stepper Motor	
Clock Speed	14 MHz	Vin	5V	IN1	PH0
Measurement, UART, Status LEDs	PN0, PF0, PF4	GND	GND	IN2	PH1
UART Baud Rate	115200bps	SDA	PB3	IN3	PH2
Start Pushbutton (onboard)	PJ0	SCL	PB2	IN4	PH3
		I2C Slave Address	0x29	V+	5V
		Interface Protocol and Data-Ready method	I2C, polling	V-	GND

Detailed Description (Distance Measurements)

The main source of distance measurements for this device comes from the VL53L1X ToF sensor. It has two main components: an emitter and a receiver. The ToF sensor emits a beam of light which gets reflected by an object or surface in proximity, before returning back to the receiver. The time difference between the emission and receiving of the light beam is used to determine the physical distance between the sensor and the surface of interest. Mathematically, the formula used to calculate this distance measurement is:

$$D = \frac{\Delta t}{2} \times \text{speed of light}$$

, where the speed of light is estimated to be 3.0×10^8 m/s. After this measurement is conducted, the ToF sensor conducts several operations to get a digital representation of the distance measurement. These include transduction, conditioning, and ADC (analog-to-digital) conversion [2]. This information then gets sent to the microcontroller through I2C serial communication.

During startup, the microcontroller first performs all required hardware initialization, including clock setup, SysTick timing, LED initialization, UART, and I2C module to interface with the ToF sensor. After hardware initialization, the program calls the function `WaitForSensorBoot()`, which continuously checks the device boot state until the sensor reports that it is ready to acquire data. This step is critical as it ensures that the ToF sensor is fully powered and responsive before any attempt is made to read or configure from it. An LED flash is then used as a visual indication that the bootup/initialization process of the sensor has completed successfully.

Once the sensor has booted, the sensor begins active distance sensing by calling the function `Sensor_StartRanging()`. This routine begins by clearing any pending sensor interrupts and starts continuous ranging with the function `VL53L1X_StartRanging(dev)`. In this specific design, the

measured distance is stored in millimetres, which is consistent with the use of the `uint16_t distance_mm[][]` array and the sensor readout function. The actual data acquisition component of each distance sample occurs in the `ReadDistanceMeasurement()` function. When this function begins, the code turns on an onboard LED indicator (PF4) to show that a measurement is in progress. It then repeatedly polls the sensor's data-ready status to determine when a new measurement is available. Once the sensor signals that data is ready, the program reads the measured distance using `VL53L1X_GetDistance(dev, &distance)`, then clears the sensor interrupt with `VL53L1X_ClearInterrupt(dev)` so that the next measurement cycle can begin properly. The LED is then turned off, and the measured distance value is returned to the calling routine. This sequence ensures that every stored reading is a fresh and valid sensor measurement.

Each reading is associated with a specific scan angle inside `PerformSingleScanStep()`. First, the angle is calculated as `index * ANGLE_INCREMENT_DEG`, where the increment is 11.25° . The code then calls `ReadDistanceMeasurement()` to obtain the radial distance at that angular position. Both the angle and measured distance are saved using `SaveScanPoint(scan, index, angle, distance)`, which stores the raw scan data into the `angle_deg` and `distance_mm` arrays. After this, the stepper motor rotates clockwise by one 11.25° increment so that the next reading will correspond to the next direction in space. In this way, one complete scan consists of 32 individual measurements covering 360° , and the procedure is repeated across multiple scan passes.

After all scans are collected, the raw polar data is converted to Cartesian coordinates using the function `ConvertScanToCartesian()`. For each stored point in the array, the measured angle in degrees is converted to radians, which is converted into appropriate XY data points using cosine and sine. Displacement was implemented manually by moving the scanning device a fixed distance between full 360° scans. Each scan was then assigned a constant offset based on scan number and scan spacing so that multiple YZ slices could be separated along the orthogonal axis and reconstructed as a 3D hallway model. This conversion is the final step before the system begins UART transmission to MATLAB for data visualization.

Detailed Description (Visualization)

After the microcontroller finishes all scans, the structured coordinate data is sent to MATLAB through UART serial communication for visualization. The MATLAB script opens the correct serial port (COM4 for this design) at a baud rate of 115200, which matches the UART settings used in the embedded system code in Keil. To keep the communication organized, MATLAB uses a simple "handshaking" process with the microcontroller. It waits until the character 's' is received, which symbolizes the microcontroller is ready to send data. MATLAB then sends back an 's' as a way of acknowledgement. From here, the coordinate transmission can start. The process is conducted twice for the X and Y data, ensuring that the values are received in the right order.

Once the data is collected, the X and Y arrays are reshaped so that each row represents one full scan of the desired environment. MATLAB then assigns each scan a different x-offset so the three scans can be displayed as separate slices in space rather than all overlapping at the same location. In this plot, the offset represents hallway depth, while the received coordinates are used for the vertical and horizontal axes. The 3D plot is created using `scatter3()` to show each measured point and `plot3()` to connect points within the same scan. Additional lines are also drawn between corresponding points of different scans, which helps form a clearer wireframe

view of the hallway. A red marker is placed at the origin to represent the sensor's starting location.

Limitations

The MSP432E401Y includes a hardware floating point unit (FPU) which enables the use of single-precision floating point operations. However, in this program, sine and cosine are used to convert the polar measurements of the ToF sensor into cartesian coordinates. This creates additional complexity which may add inaccuracies in computation and calculations. Additionally, when multiple scans are being conducted, this results in increased execution time compared to integer calculations, and rounding errors due to finite precision (32-bit float).

The VL53L1X outputs distance readings digitally rather than through an analog voltage, so ADC quantization from the MSP432E401Y does not apply. Instead, the relevant quantization comes from the sensor's own reported distance format. Since the measured distance is given in discrete 1 mm steps [1], the smallest change the sensor can represent is 1 mm. Therefore, the maximum quantization error is half of one step size (+/- 0.5mm).

The maximum standard UART communication rate supported by the PC serial interface is 128000bps. This was confirmed by going to Device Manager, and looking at the BPS information under the XDS110 Class Application/User UART port. However, the speed implemented for this device was 115200bps. This speed is fast enough to transmit the required scan data efficiently without increasing the risk of communication errors.

The communication between the microcontroller and the ToF sensor was implemented using I2C. In this specific design, the MSP432E401Y acts as the I2C master, while the VL53L1X operates as the slave with unique address 0x29. Communication between the microcontroller and the VL53L1X ToF sensor was implemented using the I2C protocol. According to the datasheet, the maximum transmission speed of the ToF sensor is 50Hz.

The primary limitation with regards to system speed would be the speed at which the stepper motor rotates. The main factor that influenced the speed of the motor was the delay between each phase of rotation using SysTick. The main thing to note here is that if the delay between the phases were too short, the motor would not have enough time to respond to each rotation sequency properly. This leads to missed steps and stalling of the motor, creating inaccurate rotations and random instability. As a result, the motor speed had to be chosen carefully to balance fast scanning with reliable operation. After several rounds of testing and reducing the delay to its limit, it was concluded that the CW delay was set to be 50000 SysTick cycles and the CCW delay is 30000 SysTick cycles. Two additional factors also limit overall system speed. First, the VL53L1X ToF sensor requires finite time to complete each ranging cycle, so the microcontroller must wait until new data is ready before recording the next measurement. Second, UART communication to the PC introduces transmission delay, since all coordinate values must be serially sent to MATLAB at 115200 bps before visualization can occur.

To configure the assigned bus speed for this device (14MHz), the system clock was derived from the following formula in the file PLL.h:

$$f_{SYSCLK} = \frac{480MHz}{PSYSDIV + 1} = 14MHz$$

For reference, the parameter PSYSDIV is a divider value that determines how much the 480MHz PLL output is reduced to produce the final system clock. To achieve the assigned bus speed of 14MHz, the equation was rearranged and isolated for the only unknown:

$$PSYSDIV = \frac{480MHz}{14MHz} - 1 \approx 33.286 \approx \mathbf{33}$$

As shown above, since PSYSDIV must be an integer, the closest integer value was selected to be 33. Plugging this value back into the original formula,

$$f_{SYSCLK} = \frac{480MHz}{33 + 1} \approx 14.12MHz$$

To verify the configured bus speed, a GPIO pin (PM0) was toggled using a known SysTick delay of 14000000 clock cycles. Thus, this delay corresponds to approximately 1 second per toggle, resulting in a measured frequency of approximately 0.5Hz.

This system uses onboard LEDs controlled by the MSP432E401Y to indicate various operational states. In this specific system:

- PN0 (LED D2) – Measurement Status
 - Turns ON while the VL53L1X sensor is acquiring a distance measurement and turns OFF when the measurement is done. This results in a rapid blinking during scanning.
- PF0 (LED D4) – UART Transmission Status
 - Turns ON when data is being transmitted from the microcontroller to MATLAB via UART. In the current implementation, it activates during Y and Z data transfers.
- PF4 (LED D3) – Scan Indicator Status
 - Turns ON during a full 360-degree scan and remains ON until the stepper motor rotates and gathers all measurements for one scan.

Application Example

One practical real-world application of this device is indoor mapping for a robot vacuum. A robot vacuum can use spatial data about walls, corners, furniture edges, and hallway openings to better understand its environment and improve navigation. For this specific purpose, the device takes vertical distance slices in the YZ plane, while manual displacement between scans defines the X direction. The resulting 3D model can therefore be used as a simple prototype for low-cost indoor mapping and obstacle awareness in autonomous cleaning systems. In this project particularly, the coordinate system is defined such that Y represents vertical distance, Z represents horizontal distance within each scan slice, and X represents hallway depth/displacement between scans.

Instructions

Below are a clear set of steps to setup and use this device correctly:

- 1) Plug in the microcontroller into the user's laptop and find the respective COM port. To do this, go to the search bar on the Windows dashboard, and search 'Device Manager'. Once

that is opened, click on the 'Ports (COM and LPT)' tab and note the value (#) inside the brackets of 'XDS110 Class Application/User UART (COM #)'. This is the COM port to use in the future steps.

- 2) Open the MATLAB script and change the first line of code (port = "COM#") to the user's port number. After this step, serial communication should be ready to run.
- 3) Setup the circuit by following the pins outlined in [Device Characteristics Table](#).
- 4) Open the Keil code and on the top dashboard, click Translate→Build→Load. Then, on the physical microcontroller, click the RESET button on the top right of the device.
- 5) Run the MATLAB script and perform scans using the onboard button PJ0, which is on the upper left side of the microcontroller.

Additionally, below are a set of potential issues the user may run into when starting up this device for use and the steps required to troubleshoot them:

- 1) Verify Hardware Wiring. Before running the software, ensure all hardware components are wired correctly according to the [Device Characteristics Table](#). Specifically, verify that:
 - a. VL53L1X ToF sensor is connected to 5V, GND, PB3 (SDA), and PB2 (SCL) on the microcontroller
 - b. Stepper motor driver inputs on the ULN2003 Driver Board are connected to PH0, PH1, PH2, and PH3
 - c. The sensor is properly screwed and mounted so that it rotates together with the motor shaft
 - d. The wiring is oriented so that it does not interfere with motor rotation
 - e. The microcontroller USB cable is plugged into the laptop
- 2) MATLAB does not receive data
 - a. Verify that the COM number and baud rate are the same on the MATLAB program and Keil configuration.
 - b. Confirm that the microcontroller is powered through USB and that the program has been successfully built and loaded onto the board.
 - c. Make sure the MATLAB script is run before pressing the onboard pushbutton PJ0, so MATLAB is already waiting for the incoming UART data.
- 3) Stepper motor does not rotate properly
 - a. Verify that the ULN2003 driver board inputs are connected correctly to PH0, PH1, PH2, and PH3.
 - b. Check that the motor power and ground connections are secure and that the motor cable is fully inserted into the driver board.
 - c. Ensure the sensor and motor shaft are mechanically aligned so the sensor rotates together with the motor.
 - d. If the motor vibrates but does not turn, recheck the wiring order of the control pins, since an incorrect phase sequence can prevent proper stepping.
 - e. Make sure no wires or external objects are blocking the motor's rotation path.

Expected Output

The expected output can be observed by looking at a side-by-side comparison from a location on campus (ITB-143 hallway) and the corresponding MATLAB output.



Figure 3: Real-world image of hallway (start)



Figure 4: Real-world image of hallway (end)

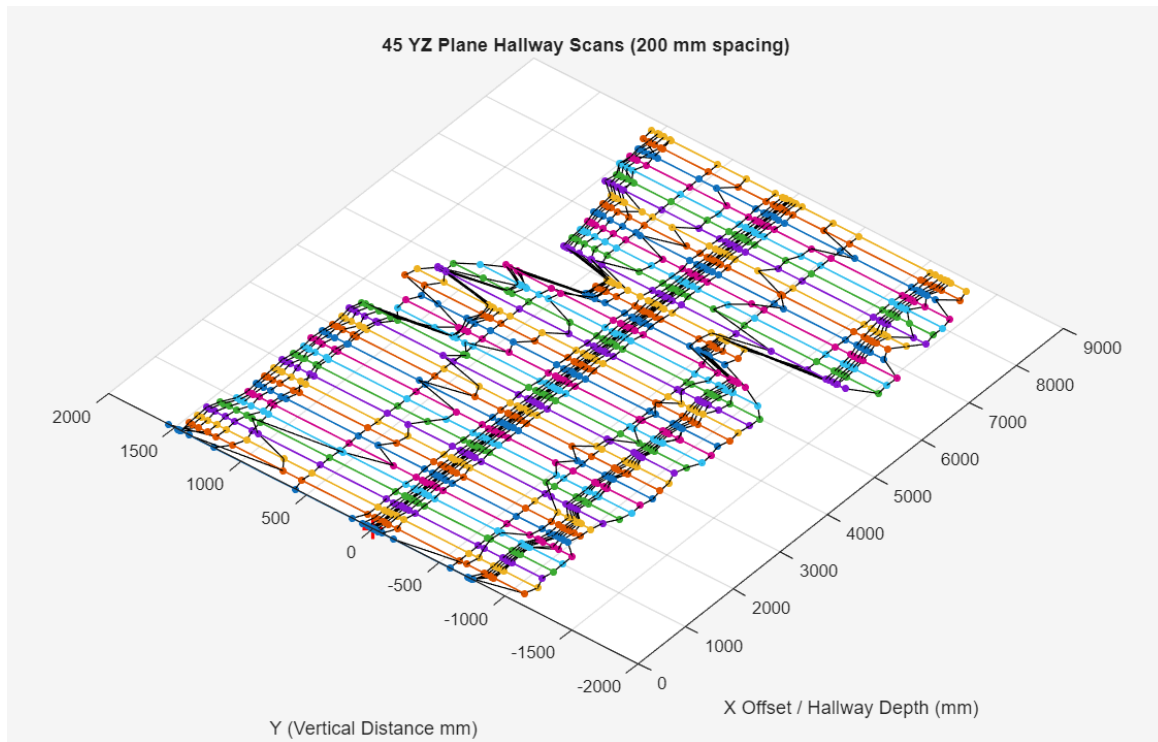


Figure 5: MATLAB Plot of hallway (Bird's Eye View)

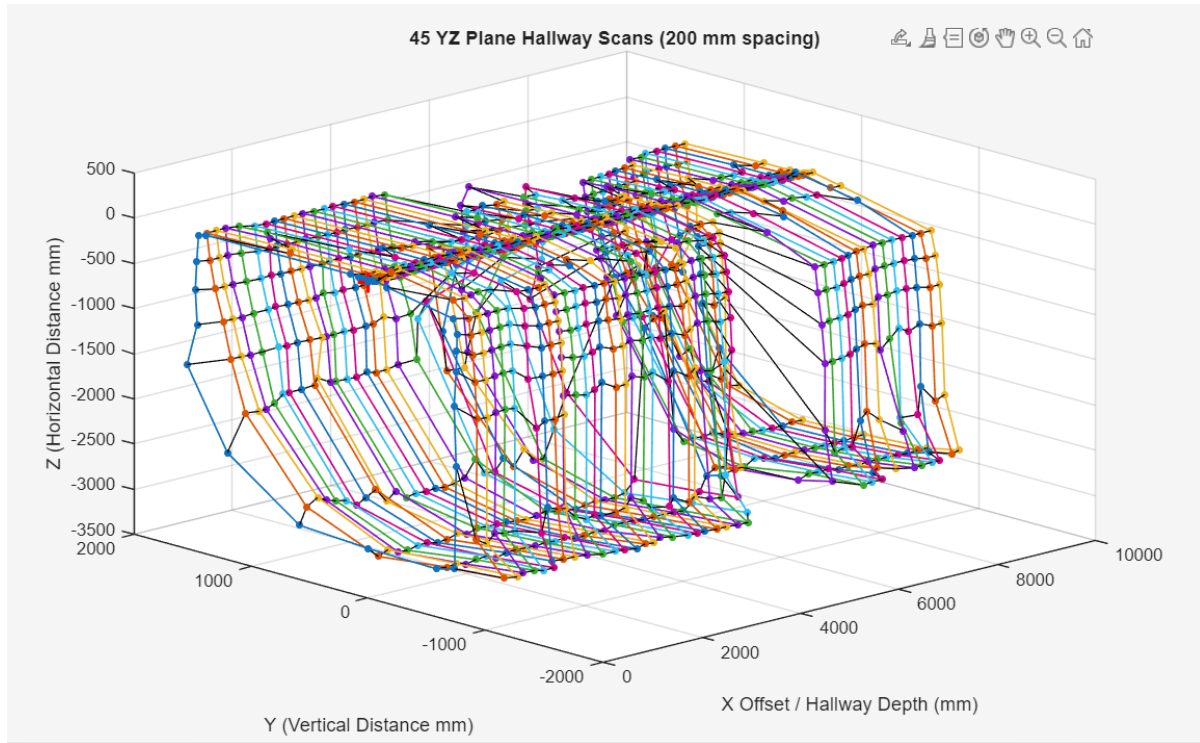


Figure 6: MATLAB Plot of hallway (side view)

As shown by the following images, the plotted MATLAB output closely matches the framework of the physical location chosen. The hallway has two main features: a set of 3 garbage cans on the left-hand side of the hallway, and an “hourglass” shape hallway. Viewing Figure 5 and 6, both of these images closely resemble the hallway, with the desired obstructions correctly visualized by the device.

Circuit Schematic

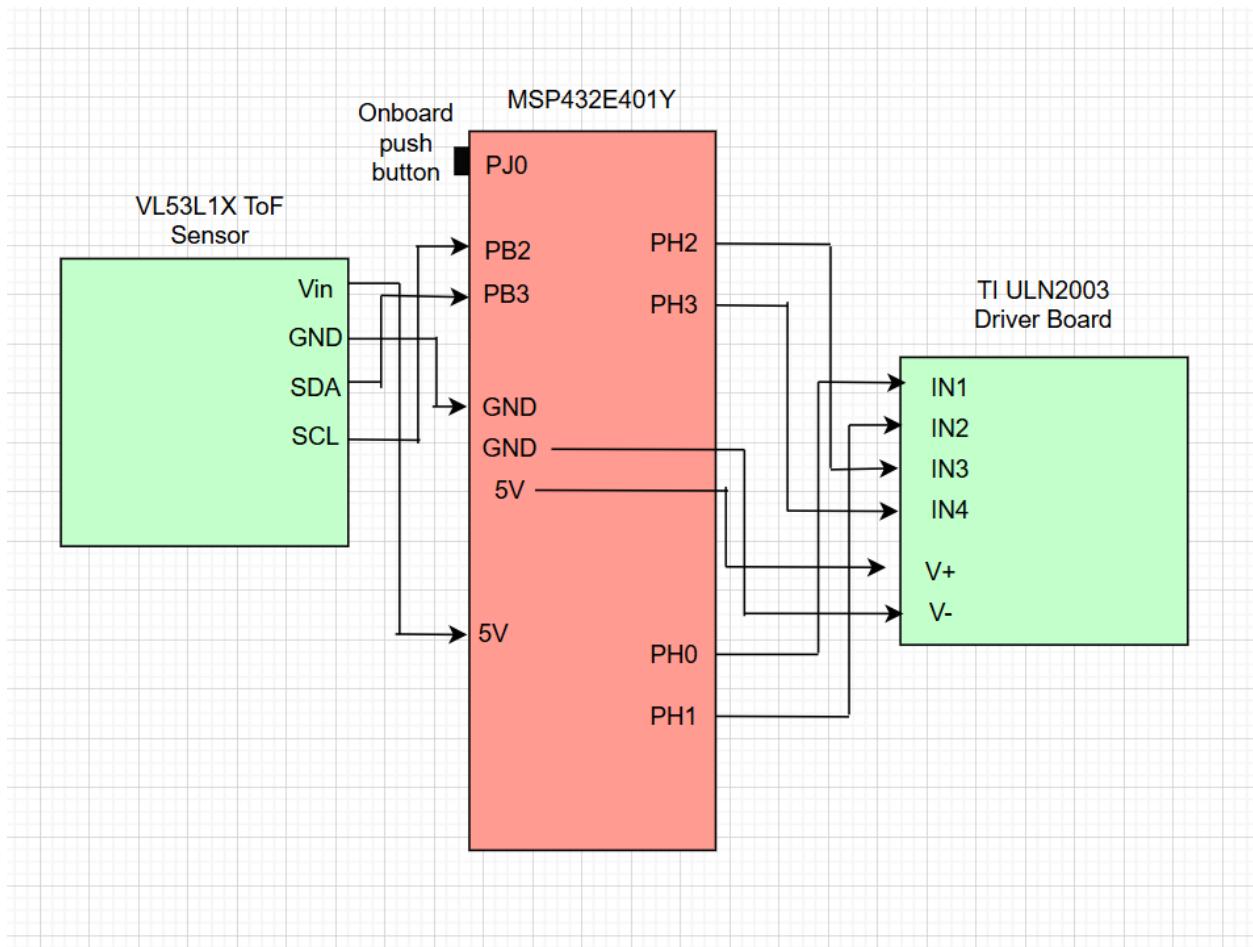


Figure 7: Circuit Schematic of main wiring for the device.

Programming Logic Flowchart

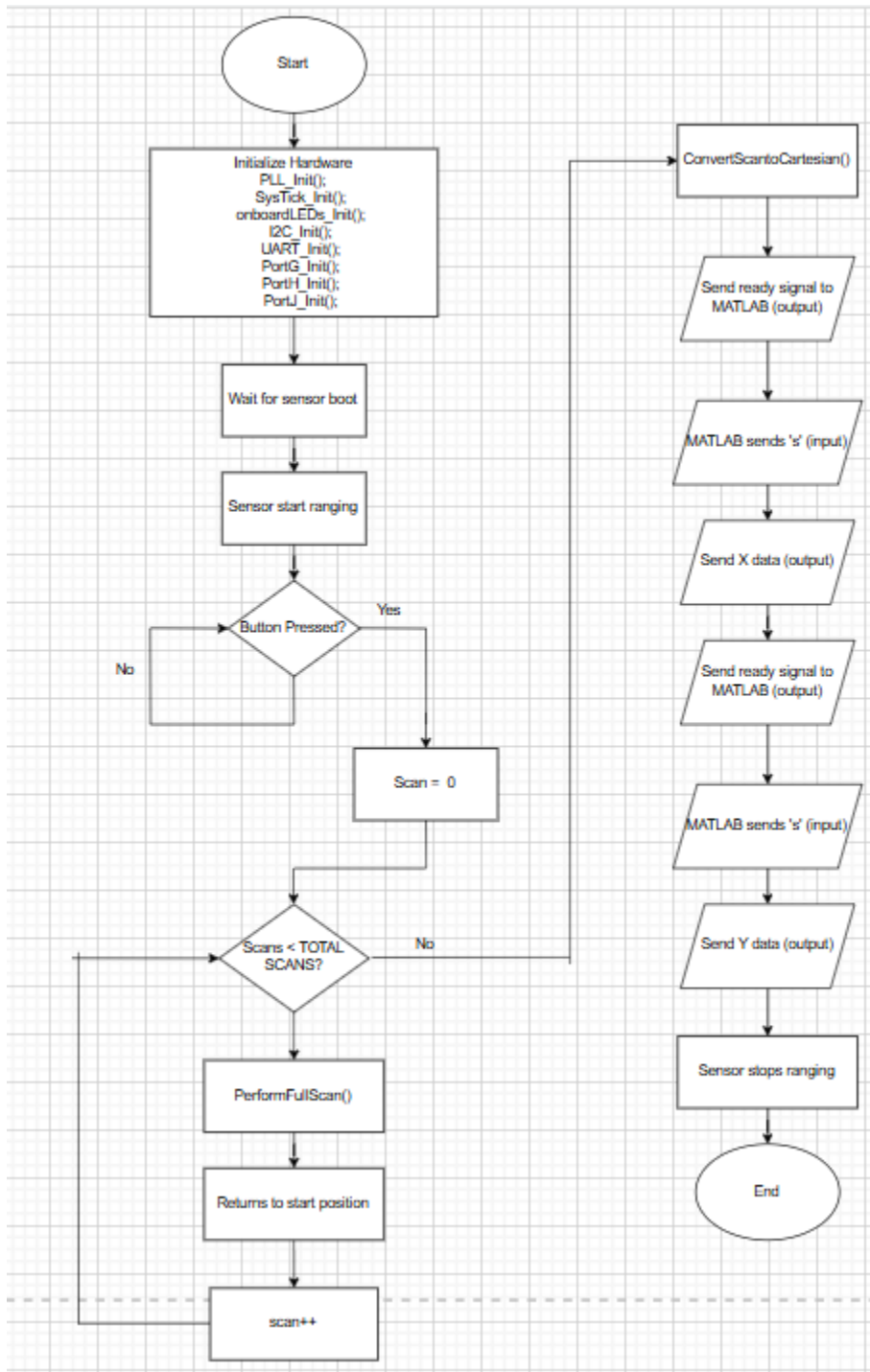


Figure 8: Programming Logic flowchart, mainly focusing on sensor operation and data transmission.

Separate flowchart for PerformFullScan():

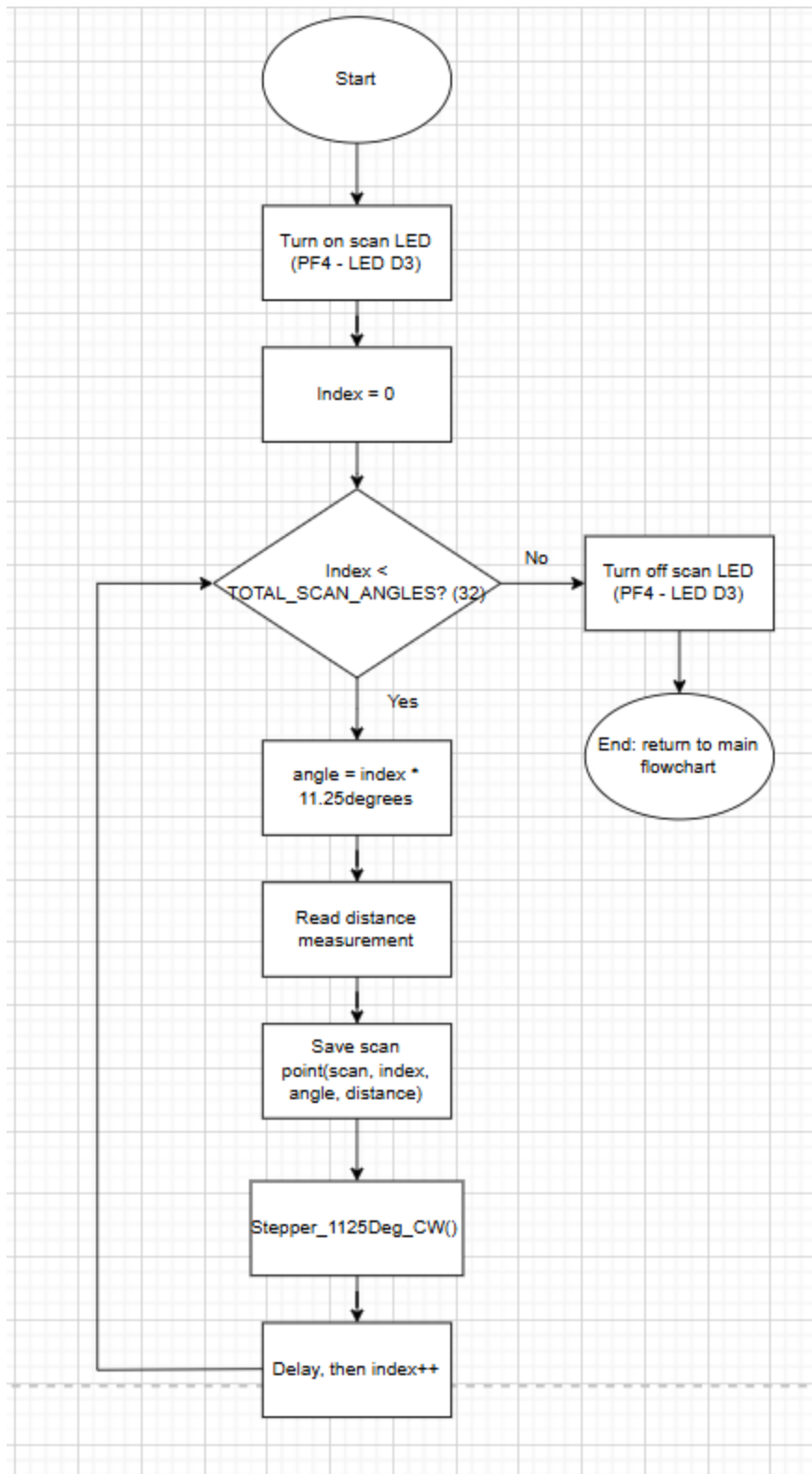


Figure 9: Separate flowchart for PerformFullScan(), relating to the stepper motor operation and LED response.

MATLAB Flowchart:



Figure 10: MATLAB flowchart illustrating the process to plot XY measurements.

References

- [1] STMicroelectronics, “A new generation, long distance ranging Time-of-Flight sensor based on ST’s FlightSense™ technology” VL53L1X datasheet, Feb. 2018 [Revised Nov. 2018].
- [2] Y. Haddara, M. Elamien, and S. Athar, “Transduction & Analog Signal Acquisition,” *Computer Engineering 2DX3/2DX4*, Jan. 2026.
https://avenue.cllmcmaster.ca/content/enforced/731113-COMPENG_2DX3_yaser_2261/Lecture%20materials/2022%20DX%20W2%20Transduction%20and%20analog%20acquisition.pdf?ou=731113 (accessed Apr. 6, 2026).
- [3] Texas Instruments, “MSP432E401Y SimpleLink™ Ethernet Microcontroller” MSP432E401Y datasheet, Oct. 2017 [Revised Nov. 2019].